CME 213 — Milestone 2

CUDAsearch: Distributed GPU-Accelerated Dense Vector Similarity Search

Ava Kouhana · Ethan Cohen

1 Project scope and problem definition

We are building **CUDAsearch**, a distributed exhaustive maximum inner product search (MIPS) system. Given a database $X \in \mathbb{R}^{N \times d}$ of pre-computed embeddings and a batch of queries $Q \in \mathbb{R}^{B \times d}$, the system returns for each query the indices and scores of its top-k nearest neighbors under the inner product $\langle q, x_i \rangle$. When all vectors are L_2-normalized, this inner product is exactly the cosine similarity, which is the standard metric used in semantic search and retrieval-augmented generation (RAG).

Inputs: a row-major float32 database of shape (N, d), a query batch (B, d), and the cutoff k. **Outputs:** two arrays of shape (B, k) containing the top-k indices and scores, sorted in descending order. **Assumptions:** vectors are L2-normalized at load time, and all computation is in FP32 unless explicitly quantized to INT8.

Success criteria. A satisfying outcome delivers (i) a correct CPU baseline against synthetic ground truth; (ii) a from-scratch CUDA kernel achieving significant speedups (several×) over the single-threaded CPU baseline on SIFT1M; (iii) a shared-memory tiled kernel providing a further improvement over the naive kernel; (iv) an INT8 quantized variant retaining high Recall@10 against the FP32 kernel; (v) multi-GPU strong scaling with high efficiency; and (vi) a roofline and arithmetic-intensity analysis that explains the measured performance.

2 Survey of related work

Several existing systems address large-scale vector similarity search, both on CPU and GPU.

The closest reference to our work is **FAISS** [1], in particular the `IndexFlatIP`, which performs exact (brute-force) maximum inner product search (MIPS). This serves as our main baseline for both correctness and performance.

Other systems focus on reducing computation by approximating the search. For example, **ScaNN** [2] improves CPU performance using quantization techniques, while graph-based methods such as **HNSW** [3] and **DiskANN** [4] achieve sublinear query time by exploring a graph structure instead of scanning the full dataset.

Libraries like cuBLAS and **CUTLASS** provide optimized matrix multiplication kernels, which are directly relevant since our problem can be formulated as a matrix multiplication (see Section 4). In particular, our tiled kernel builds on these ideas to improve memory efficiency.

Positioning of our work. Our goal is to build a clear and modular implementation that highlights the trade-offs between different design choices (naive vs tiled kernels, FP32 vs INT8, single vs multi-GPU). FAISS `IndexFlatIP` and cuBLAS `sgemm` could potentially serve as upper-bound reference points in our experiments.

3 Computational challenges

Memory-bound regime. With d elements, an inner product performs $2d$ FLOPs per database vector (multiply + add) over $4d$ bytes read, giving an arithmetic intensity of 0.5 FLOP/byte which is far below the ridge of modern GPUs. The interpretation is that the computation is not limited by how fast the GPU can compute, but by how fast it can read data from memory. Our goal is to confirm this with our CPU baseline (parallelized with OpenMP). Hopefully, performance will saturate after a small number of threads, indicating a memory bandwidth bottleneck. Mitigation strategies will be to process multiple queries at once (batching) to reuse data and use INT8 quantization to reduce memory traffic.

Top-k selection on GPU. A global sort of scores is wasteful when $k \ll N$. As a solution, we are therefore thinking about maintaining a small heap of size k in registers or shared memory.

MPI overhead at small scale. In a multi-GPU setup, each GPU computes partial results that must be combined. Communication (broadcast + gather) can dominate runtime when workloads are small. Our plan is to identify the regime where distributed execution is beneficial.

4 CUDA implementation plan

The core computation is a matrix multiplication: $S = QX^\top$ where each element S_{ij} is the similarity between query i and database vector j. Each row of Q is compared with all rows of X, which corresponds to many dot products computed in parallel. We implement three progressively optimized kernels:

(i) Naive kernel.

The score matrix $S \in \mathbb{R}^{B \times N}$ has $B \times N$ entries, where each entry $S_{b,i} = \langle q_b, x_i \rangle$ corresponds to exactly one dot product, so we spawn exactly $B \times N$ threads. Top-k selection is then performed on the CPU after transferring the full score matrix back to host.

Limitation: the query vector q_b is re-fetched from global memory by every thread block with no reuse across blocks; the tiled kernel addresses this by loading query tiles into shared memory and reusing them across all threads processing database rows in that tile.

(ii) Tiled kernel with shared memory.

Shared memory is much faster than global memory, so reusing data significantly improves performance. We reorganize computation into tiles:

- Blocks of Q and X are loaded into shared memory
- Threads reuse this data to compute multiple dot products

(iii) INT8 quantized kernel.

We compress database vectors using 8-bit integers to reduce memory traffic. However the trade-off is that it requires dequantization during computation and may reduce accuracy. For the evaluation, we will compare speed and recall@k against FP32.

5 MPI implementation plan

We distribute the database across multiple GPUs. Each GPU is responsible for a subset of the database. For each query batch, the query is broadcast to all GPUs. Then, each GPU computes similarities on its local data and keeps its local top-k. Finally, Results are sent back and merged into a global top-k.

Similarity computations are independent across data partitions, making the problem naturally parallel. However, the final merge step can become a bottleneck if not implemented efficiently. We are thinking about parallelizing merging across queries using OpenMP.

6 Profiling and performance analysis plan

We evaluate performance along several dimensions:

- **Kernel comparison.** Compare CPU, naive CUDA, tiled CUDA, and INT8 kernels in terms of latency and throughput for different k and batch sizes B.
- **Scaling with data.** Study how performance varies with database size N and vector dimension d.
- **Multi-GPU scaling.** Measure strong scaling (fixed problem size) and weak scaling (data increases with number of GPUs).
- **Accuracy.** Measure recall@10 of the INT8 kernel compared to the FP32 version.
- **Baseline comparison.** If possible, compare end-to-end performance against FAISS `IndexFlatIP`.

7 Timeline

Week of	Tasks
Apr 28	CPU baseline; Implement naive CUDA kernel; validate recall against CPU.
May 5	Shared-memory tiled kernel. INT8 quantization + CPU INT8 reference.
May 12	MPI sharding skeleton; single-query multi-GPU correctness.
May 19	Distributed top-k merge with OpenMP; strong/weak scaling experiments.
May 26	Benchmark sweeps; FAISS comparison; write M4 progress report.
Jun 2	Final benchmarks; figure generation; draft final report.
Jun 8	Submit final report and code.

References

[1] J. Johnson, M. Douze, H. Jégou, Billion-scale similarity search with GPUs, *IEEE Trans. Big Data*, 7(3), 2021.

[2] R. Guo et al., Accelerating large-scale inference with anisotropic vector quantization, *ICML*, 2020.

[3] Y. A. Malkov, D. A. Yashunin, Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs, *IEEE TPAMI*, 42(4), 2020.

[4] S. Subramanya et al., DiskANN: Fast accurate billion-point nearest neighbor search on a single node, *NeurIPS*, 2019.